

NEURAL LOGIC MACHINES

Honghua Dong^{*1}, Jiayuan Mao^{*1}, Tian Lin², Chong Wang³, Lihong Li², and Denny Zhou²

¹ ITCS, IIS, Tsinghua University {dhh14, mjoy14}@mails.tsinghua.edu.cn

² Google Inc. {tianlin, lihong, dennyzhou}@google.com

³ ByteDance Inc. chong.wang@bytedance.com

ABSTRACT

We propose the Neural Logic Machine (NLM), a neural-symbolic architecture for both inductive learning and logic reasoning. NLMs exploit the power of both neural networks—as function approximators, and logic programming—as a symbolic processor for objects with properties, relations, logic connectives, and quantifiers. After being trained on small-scale tasks (such as sorting short arrays), NLMs can recover lifted rules, and generalize to large-scale tasks (such as sorting longer arrays). In our experiments, NLMs achieve perfect generalization in a number of tasks, from relational reasoning tasks on the family tree and general graphs, to decision making tasks including sorting arrays, finding shortest paths, and playing the blocks world. Most of these tasks are hard to accomplish for neural networks or inductive logic programming alone. ¹

1 INTRODUCTION

Deep learning has achieved great success in various applications such as speech recognition (Hinton et al., 2012), image classification (Krizhevsky et al., 2012; He et al., 2016), machine translation (Sutskever et al., 2014; Bahdanau et al., 2015; Wu et al., 2016; Vaswani et al., 2017), and game playing (Mnih et al., 2015; Silver et al., 2017). Starting from Fodor & Pylyshyn (1988), however, there has been a debate over the problem of systematicity (such as understanding recursive systems) in connectionist models (Fodor & McLaughlin, 1990; Hadley, 1994; Jansen & Watter, 2012).

Logic systems can naturally process symbolic rules in language understanding and reasoning. Inductive logic programming (ILP) (Muggleton, 1991; 1996; Friedman et al., 1999) has been developed for learning logic rules from examples. Roughly speaking, given a collection of positive and negative examples, ILP systems learn a set of rules (with uncertainty) that entails all of the positive examples but none of the negative examples. Combining both symbols and probabilities, many problems arose from high-level cognitive abilities, such as systematicity, can be naturally resolved. However, due to an exponentially large searching space of the compositional rules, it is difficult for ILP to scale beyond small-sized rule sets (Dantsin et al., 2001; Lin et al., 2014; Evans & Grefenstette, 2018).

To make the discussion concrete, let us consider the classic blocks world problem (Nilsson, 1982; Gupta & Nau, 1992). As shown in Figure 1, we are given a set of blocks on the ground. We can move a block x and place it on the top of another block y or the ground, as long as x is *moveable* and y is *placeable*. We call this operation $\text{MOVE}(x, y)$. A block is said to be *moveable* or *placeable* if there are no other blocks on it. The ground is always *placeable*, implying that we can place all blocks on the ground. Given an initial configuration of blocks world, our goal is to transform it into a target configuration by taking a sequence of MOVE operations.

Although the blocks world problem may appear simple at first glance, four major challenges exist in building a learning system to automatically accomplish this task:

1. The learning system should recover a set of lifted rules (*i.e.*, rules that apply to objects uniformly instead of being tied with specific ones) and generalize to blocks worlds which contain more blocks than those encountered during training. To get an intuition on this, we refer the readers who are not familiar with the blocks world domain to the task of learning to sort arrays (e.g.,

^{*}indicates equal contribution. This work was done when the first two authors were interns at Google.

¹Project page: <https://sites.google.com/view/neural-logic-machines>.

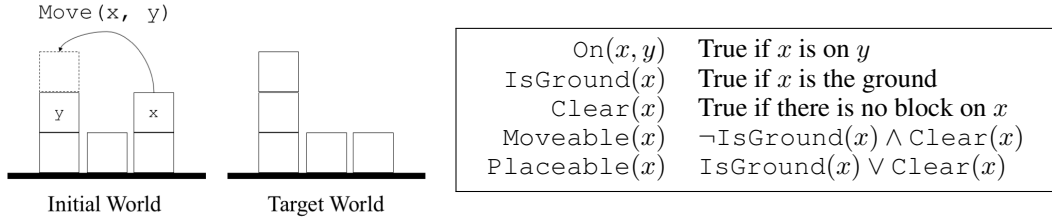


Figure 1: (Left) A graphical illustration of the blocks world. Given an initial and a target worlds, the agent is required to move blocks to transform the initial configuration to the target one. (Right) A set of sentences used throughout the paper to define the blocks world.

Vinyals et al., 2015), where recurrent neural networks fail to generalize to arrays which are even just slightly longer than those for training.

2. The learning system should deal with high-order relational data and quantifiers, which goes beyond the scope of typical graph-structured neural networks (Kipf & Welling, 2017). For example, to apply the transitivity rule of a relation r , i.e. $r(a, c) \leftarrow \exists b \, r(a, b) \wedge r(b, c)$, we need to jointly inspect three objects (a, b, c) .
3. The learning system should scale up w.r.t. the complexity of the rules.² Existing logic-driven approaches such as traditional ILP methods suffer an exponential computational complexity w.r.t. the number of logic rules to be learned (Dantsin et al., 2001; Lin et al., 2014; Evans & Grefenstette, 2018).
4. The learning system should recover rules based on a minimal set of learning priors. In contrast, traditional ILP methods usually require hand-coded and task-specific rule templates to restrict the size of searching spaces (Evans & Grefenstette, 2018).

In this paper, we propose Neural Logic Machines (NLMs) to address the aforementioned challenges. In a nutshell, NLMs offer a neural-symbolic architecture which realizes Horn clauses (Horn, 1951) in first-order logic (FOL). The key intuition behind NLMs is that logic operations such as logical ANDs and ORs can be efficiently approximated by neural networks, and the wiring among neural modules can realize the logic quantifiers.

The rest of the paper is organized as follows. We first revisit some useful definitions in symbolic logic systems and define our neural implementation of a rule induction system in Section 2. As a supplementary, we refer interested readers to Appendix A for implementation details. In Section 3 we evaluate the effectiveness of NLM on a broad set of tasks ranging from relational reasoning to decision making. We discuss related works in Section 4, and conclude the paper in Section 5.

2 NEURAL LOGIC MACHINES (NLM)

The NLM is a neural realization of logic machines (under the Closed-World Assumption³). Given a set of base predicates, grounded on a set of objects (the *premises*), NLMs sequentially apply first-order rules to draw *conclusions*, such as a property about an object. For example, in the blocks world, based on premises $\text{IsGround}(u)$ and $\text{Clear}(u)$ of object u , NLMs can infer whether u is moveable.

Internally, NLMs use tensors to represent logic predicates. This is done by grounding the predicate as *True* or *False* over a fixed set of objects. Based on the tensor representation, rules are implemented as neural operators that can be applied over the premise tensors and generate conclusion tensors. Such neural operators are probabilistic, lifted, and able to handle relational data with various orders (i.e., operating on predicates with different arities).

2.1 LOGIC PREDICATES AS TENSORS

We adopt a probabilistic tensor representation for logic predicates. Suppose we have a set of objects $\mathcal{U} = \{u_1, u_2, \dots, u_m\}$. A predicate $p(x_1, x_2, \dots, x_r)$, of *arity* r , can be grounded on the object set $\mathcal{U}$ (informally, we call it $\mathcal{U}$ -*grounding*), resulting in a tensor $p^{\mathcal{U}}$ of shape $[m^r] \triangleq [m, m-1, m-2, \dots, m-r+1]$, where the value of each entry $p^{\mathcal{U}}(u_{i_1}, u_{i_2}, \dots, u_{i_r})$ of the tensor represents whether p is *True* under the grounding that $x_1 = u_{i_1}, x_2 = u_{i_2}, \dots, x_r = u_{i_r}$. Here, we restrict that the grounded objects of all x_i 's are mutually exclusive, i.e., $i_j \neq i_k$ for all pairs of

²For a concrete example in the blocks world domain, please refer to Appendix E.

³https://en.wikipedia.org/wiki/Closed-world_assumption

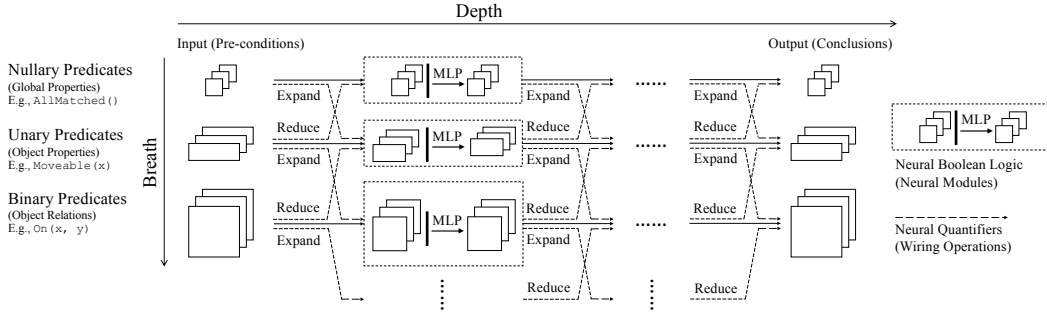


Figure 2: An illustration of Neural Logic Machines (NLM). During forward propagation, NLM takes object properties and relations as input, performs sequential logic deduction, and outputs conclusive properties or relations of the objects. Implementation details can be found in Section 2.3.

indices j and k . This restriction does not limit the generality of the representation, as the “missing” entries can be represented by the $\mathcal{U}$ -grounding of other predicates with a smaller arity. For example, for a binary predicate p , the grounded values of the $p^{\mathcal{U}}(x, x)$ can be represented by the $\mathcal{U}$ -grounding of a unary predicate $p'(x) \triangleq p(x, x)$.

We extend this representation to a collection of predicates of the same arity. Let $C^{(r)}$ be the number of predicates of arity r . We stack the $\mathcal{U}$ -grounding tensors of all predicates as a tensor of shape $[m^x, C^{(r)}] \triangleq [m, m-1, m-2, \dots, m-r+1, C^{(r)}]$, where the last dimension corresponds to the predicates. Intuitively, a group of $C^{(1)}$ unary predicates grounded on m objects can be represented by a tensor of shape $[m, C^{(1)}]$, describing a group of “properties of objects”, while a $[m, m-1, C^{(2)}]$ -shaped tensor for $C^{(2)}$ binary predicates describes a group of “pairwise relations between objects”. In practice, we set a maximum arity B for the predicates of interest, called the *breadth* of the NLM.

In addition, NLMs take a probabilistic view of predicates. Each entry in $\mathcal{U}$ -grounding tensors takes value from $[0, 1]$, which can be interpreted as the probability being `True`. All premises, conclusions, and intermediate results in NLMs are represented by such probabilistic tensors. As a side note, we impose the restriction that all arguments in the predicates can only be variables or objects (*i.e.*, constants) but not function symbols, which follows the setting of *Datalog* (Maier & Warren, 1988).

2.2 LOGIC RULES AS NEURAL OPERATORS

Our goal is to build a neural architecture to learn rules that are both lifted and able to handle relational data with multiple arities. We present different modules of our neural operators by making analogies to a set of essential *meta-rules* in symbolic logic systems. Specifically, we discuss our neural implementation of (1) *boolean logic rules*, as lifted rules containing boolean operations (AND, OR, NOT) over a set of predicates; and (2) *quantifications*, which bridge predicates with different arities by logic quantifiers ($\forall$ and $\exists$).

Next, we combine these neural units to compose NLMs. Figure 2 illustrates the overall multi-layer, multi-group architecture of an NLM. An NLM has layers of *depth* D (horizontally), and each layer has $B+1$ computation units (vertically). These units operate on the tensor representations of predicates whose arities range from $[0, B]$, respectively. NLMs take input tensors of predicates (premises), perform layer-by-layer computations, and output tensors as conclusions.

As the number of layers increases, higher levels of abstraction can be formed. For example, the output of the first layer may represent `Clear`(x), while a deeper layer may output more complicated predicate like `Moveable`(x). Thus, forward propagation in NLMs can be interpreted as a sequence of rule applications. We further show that NLMs can efficiently realize a partial set of Horn clauses.

We start from the *neural boolean logic rules* and the *neural quantifiers*.

Boolean logic. We use the following symbolic meta-rule for boolean logic:

$$\hat{p}(x_1, x_2, \dots, x_r) \leftarrow \text{expression}(x_1, x_2, \dots, x_r), \quad (1)$$

where *expression* can be any *boolean* expressions consisting of predicates over *all* variables $(x_1, \dots, x_r)$ and $\hat{p}(\cdot)$ is the conclusive predicate. For example, the rule `Moveable`(x) $\leftarrow \neg \text{IsGround}(x) \wedge \text{Clear}(x)$ can be instantiated from this meta-rule.

Denote $\mathcal{P} = \{p_1, \dots, p_k\}$ as the set of $|\mathcal{P}|$ predicates appeared in *expression*. By definition, all p_i ’s have the same arity r and can be stacked as a tensor of shape $[m^x, |\mathcal{P}|]$. In Eq. 1, for